

=====

SOAR UNIFIED API - COMPREHENSIVE DOCUMENTATION

Version 1.0.0

=====

=====

TABLE OF CONTENTS

=====

- 1. Introduction
 - 1.1 Purpose
 - 1.2 Architecture Overview
 - 1.3 Technology Stack
- 2. Quick Start
 - 2.1 Prerequisites
 - 2.2 Environment Configuration
 - 2.3 Running the API
- 3. API Endpoints - Detailed Reference
 - 3.1 Alerts - Team 0: Core Data
 - 3.2 Risk Score - Team 1: Risk Scoring
 - 3.3 Patch Recommendation - Team 2: Patch Recommendation
 - 3.4 Filtering - Team 3: Log Filtering & Noise Reduction
 - 3.5 Playbooks - Team 4: Playbooks
 - 3.6 Threat Intel - Team 5: Threat Intel
 - 3.7 Health Check
 - 3.8 Root Endpoint
- 4. Data Models - Complete Reference
 - 4.1 Core Schemas
 - 4.2 Team-Specific Schemas
- 5. Integration Details
 - 5.1 External Integrations
 - 5.2 Parallel Processing
 - 5.3 Fail-Soft Architecture
- 6. Advanced Features
 - 6.1 Enrichment Pipeline
 - 6.2 CVE Extraction
 - 6.3 Risk Scoring Algorithm
 - 6.4 Playbook Decision Logic
- 7. Error Handling
 - 7.1 HTTP Status Codes
 - 7.2 Error Response Format
 - 7.3 Service Availability
- 8. Troubleshooting
 - 8.1 Common Issues
 - 8.2 Debug Information
- 9. Rate Limits & Best Practices

=====

1. INTRODUCTION

=====

1.1 PURPOSE

The SOAR (Security Orchestration, Automation and Response) Unified API provides a centralized backend for security operations. It integrates with Wazuh and OpenSearch to fetch alerts, enrich them with external threat intelligence, calculate risk scores, recommend patches, filter noise, and automate response playbooks.

1.2 ARCHITECTURE OVERVIEW

The API is organized into six functional teams:

- TEAM 0: Core Data
 - Fetches alerts from Wazuh/OpenSearch
 - Provides unified data normalization
 - Optional parallel enrichment with VT, AbuseIPDB, MISP
- TEAM 1: Risk Scoring
 - Calculates risk scores (0-100) based on multiple factors
 - Determines priority levels (LOW/MEDIUM/HIGH/CRITICAL)
 - Provides confidence metrics
- TEAM 2: Patch Recommendation
 - Analyzes alerts for vulnerable software
 - Queries NVD (National Vulnerability Database) for CVE details
 - Queries EPSS (Exploit Prediction Scoring System) for exploit likelihood
 - Provides prioritized patch recommendations
- TEAM 3: Log Filtering & Noise Reduction
 - LLM-assisted alert classification
 - Distinguishes between noise and important alerts
 - Uses rule ID, severity, statistics, and context features
- TEAM 4: Playbooks
 - Tag-based decision logic
 - Risk-score threshold evaluation
 - Automated action recommendations (QUARANTINE, BLOCK, INVESTIGATE, etc.)
- TEAM 5: Threat Intel
 - IOC lookup in VirusTotal, AbuseIPDB, MISP
 - CVE extraction from alert descriptions
 - EPSS and CVSS score retrieval
 - MISP event synchronization

1.3 TECHNOLOGY STACK

- Framework: FastAPI (Python)
- Web Server: Uvicorn
- Database: OpenSearch
- Intelligence Sources: VirusTotal, AbuseIPDB, MISP, NVD, EPSS
- HTTP Client: httpx (async)

=====

2. QUICK START

=====

2.1 PREREQUISITES

- Python 3.9+
- OpenSearch instance (for alert storage)
- MISP instance (for threat intelligence)
- VirusTotal API key
- AbuseIPDB API key (optional)

2.2 ENVIRONMENT CONFIGURATION

Set the following environment variables before starting the API:

+-----+-----+-----+		
Variable Description		
+-----+-----+-----+		
OS_HOST	OpenSearch host URL (e.g., http://localhost:9200)	
OS_PORT	OpenSearch port (default: 9200)	
OS_USER	OpenSearch username	
OS_PASS	OpenSearch password	
VT_API_KEY	VirusTotal API key	

MISP_URL	MISP instance URL	
MISP_KEY	MISP API key	
ABUSEIPDB_KEY	AbuseIPDB API key (optional)	
+-----+-----+-----+		

Example (.env file):

```
OS_HOST=http://localhost:9200
OS_PORT=9200
OS_USER=admin
OS_PASS=admin
VT_API_KEY=your_virustotal_api_key
MISP_URL=https://misp.example.com
MISP_KEY=your_misp_api_key
```

2.3 RUNNING THE API

```
# Install dependencies
pip install -r requirements.txt

# Run the server
uvicorn main:app --host 0.0.0.0 --port 8000

# Or using the module directly
python -m uvicorn main:app --reload
```

The API will be available at: <http://localhost:8000>

Auto-generated API documentation (Swagger UI):
<http://localhost:8000/docs>

3. API ENDPOINTS - DETAILED REFERENCE

3.1 ALERTS - TEAM 0: CORE DATA

ENDPOINT: GET /api/v1/alerts/

DESCRIPTION:
Fetches Wazuh alerts from OpenSearch with optional parallel enrichment.
When enrich=true, each alert is enriched concurrently against VirusTotal, AbuseIPDB, and MISP using asyncio.gather with return_exceptions=True.

QUERY PARAMETERS:

Parameter	Type	Default	Description	
limit	int	100	Number of alerts to fetch (1-1000)	
offset (alias: from)	int	0	Number of alerts to skip	
severity	int	None	Filter by Wazuh rule level (1-15)	
enrich	bool	false	Enable external intelligence	

EXAMPLE REQUEST:

```
curl -X GET "http://localhost:8000/api/v1/alerts/?limit=10&enrich=true"
```

EXAMPLE RESPONSE:

```
[
  {
    "event_id": "wazuh-alerts-2024.01.15-001",
    "source": "wazuh",
    "timestamp": "2024-01-15T10:30:00Z",
    "description": "sshd: authentication failed for root from 192.168.1.50",
  }
]
```

```

    "severity": 5,
    "host_context": {
      "hostname": "prod-web-01",
      "ip_address": "192.168.1.100",
      "mac_address": "00:11:22:33:44:55",
      "os_name": "Ubuntu 22.04 LTS"
    },
    "enrichment_data": {
      "vt_score": 45,
      "abuse_score": 72,
      "epss_score": 0.85,
      "cvss_score": 7.8,
      "risk_score": 68,
      "misp_matches": [],
      "tags": ["suspicious", "ssh", "brute-force"],
      "debug_info": {}
    },
    "raw_data": {
      "rule": {"id": 5710, "level": 5},
      "full_log": "Jan 15 10:30:00 prod-web-01 sshd[12345]: Failed password for root from 192.168.1.50 port 22 ssh2",
      "srcip": "192.168.1.50"
    }
  }
}
```

BEHAVIOR DETAILS:

- Parallel Enrichment: When enrich=true, all alerts are processed in parallel using asyncio.gather. Each alert's enrichment (VT, AbuseIPDB, MISP) also runs in parallel within each alert.
- Timeouts: Individual enrichment service calls are capped at 5 seconds.
- Fail-Soft: Uses return_exceptions=True so one failing enrichment never kills the rest of the batch.
- Safe Defaults: For internal/private IP addresses:
 - vt_score = 0 (clean)
 - abuse_score = 100 (safe)
 - enrichment is skipped
- Batch Protection: Entire enrichment process continues even if some alerts fail, returning safe defaults for failed ones.

3.2 RISK SCORE - TEAM 1: RISK SCORING

ENDPOINT: POST /api/v1/risk-score/

DESCRIPTION:

- Calculates a comprehensive risk score (0-100) for an alert based on:
- Base severity from Wazuh rule level
 - VirusTotal reputation score
 - AbuseIPDB confidence score
 - CVSS (Common Vulnerability Scoring System) score
 - EPSS (Exploit Prediction Scoring System) score

REQUEST BODY:

```
{
  "alert": UnifiedAlert
}
```

EXAMPLE REQUEST:

```
curl -X POST "http://localhost:8000/api/v1/risk-score/" \
-H "Content-Type: application/json" \
-d '{
  "alert": {
    "event_id": "alert-001",
```

```
    "source": "wazuh",
    "timestamp": "2024-01-15T10:30:00Z",
    "description": "Web shell detected",
    "severity": 12,
    "host_context": {
      "hostname": "prod-app-01",
      "ip_address": "10.0.0.50"
    },
    "raw_data": {},
    "enrichment_data": {
      "vt_score": 95,
      "abuse_score": 85,
      "cvss_score": 9.8,
      "epss_score": 0.97
    }
  }
}
```

EXAMPLE RESPONSE:

```
{
  "event_id": "alert-001",
  "risk_score": 94,
  "priority": "CRITICAL",
  "confidence": 0.85,
  "features": {
    "base_severity": 120,
    "enrichment": 149.85,
    "cvss": 9.8,
    "epss": 0.97
  }
}
```

RISK SCORING ALGORITHM:

- Base Score = severity * 10
 - Severity range: 1-15
 - Base score range: 10-150
- Enrichment Score Components:
 - CVSS contribution: cvss * 5 (max: 49)
 - EPSS contribution: epss * 50 (max: 50)
 - Abuse score contribution: abuse_score * 0.5 (max: 50)
 - VT score contribution: vt_score * 0.5 (max: 50)
- Final Score = min(base_score + enrichment_score, 100)
- Priority Mapping:
 - Score > 70: CRITICAL
 - Score > 30: HIGH
 - Score <= 30: MEDIUM
 - (Note: Score 0-30 returns LOW in current implementation)
- Confidence: Fixed at 0.85 (85%)

3.3 PATCH RECOMMENDATION - TEAM 2: PATCH RECOMMENDATION

ENDPOINT: POST /api/v1/patch/

DESCRIPTION:
Analyzes alerts to identify vulnerabilities and retrieves patch recommendations from NVD (National Vulnerability Database) and EPSS (Exploit Prediction Scoring System).

REQUEST BODY:

```
{
  "alert": UnifiedAlert
}
```

EXAMPLE REQUEST:

```
curl -X POST "http://localhost:8000/api/v1/patch/" \
-H "Content-Type: application/json" \
-d '{
  "event_id": "vuln-alert-001",
  "source": "wazuh",
  "timestamp": "2024-01-15T10:30:00Z",
  "description": "Vulnerable OpenSSL version detected - CVE-2023-1234",
  "severity": 10,
  "host_context": {
    "hostname": "prod-db-01",
    "ip_address": "192.168.2.10"
  },
  "raw_data": {},
  "enrichment_data": {
    "tags": ["vuln:CVE-2023-1234"]
  }
}'
```

EXAMPLE RESPONSE:

```
{
  "host": "prod-db-01",
  "recommendations": [
    {
      "cve": "CVE-2023-1234",
      "cvss": 9.8,
      "epss": 0.95,
      "priority": "CRITICAL",
      "action": "Apply OpenSSL 3.0.8 or later immediately"
    },
    {
      "cve": "CVE-2023-5678",
      "cvss": 7.5,
      "epss": 0.72,
      "priority": "HIGH",
      "action": "Schedule security update within 7 days"
    }
  ]
}
```

RESPONSE DETAILS:

- host: Hostname of the affected system
- recommendations: Array of patch recommendations sorted by priority
 - cve: CVE identifier (e.g., CVE-2023-1234)
 - cvss: CVSS v3.1 base score (0-10)
 - epss: EPSS probability (0-1, representing exploit likelihood)
 - priority: CRITICAL/HIGH/MEDIUM/LOW based on CVSS and EPSS
 - action: Human-readable recommended action

CVSS SCORING INTERPRETATION:

- 9.0-10.0: CRITICAL
- 7.0-8.9: HIGH
- 4.0-6.9: MEDIUM
- 0.1-3.9: LOW

EPSS SCORING INTERPRETATION:

- >= 0.9: Very likely to be exploited
- 0.5-0.89: Likely to be exploited
- 0.1-0.49: Possible exploitation
- < 0.1: Unlikely exploitation

3.4 FILTERING - TEAM 3: LOG FILTERING & NOISE REDUCTION

ENDPOINT: POST /api/v1/alerts/filter

DESCRIPTION:

LLM-assisted classification to distinguish between noise alerts and important security events. Uses multiple features including rule ID, severity, event frequency, IP reputation, and asset criticality.

REQUEST BODY:

```
{
  "alerts": [UnifiedAlert, ...]
}
```

EXAMPLE REQUEST:

```
curl -X POST "http://localhost:8000/api/v1/alerts/filter" \
-H "Content-Type: application/json" \
-d '{
  "alerts": [
    {
      "event_id": "noise-alert-001",
      "source": "wazuh",
      "timestamp": "2024-01-15T10:30:00Z",
      "description": "IDS alert",
      "severity": 3,
      "host_context": {
        "hostname": "dev-server",
        "ip_address": "192.168.10.50"
      },
      "raw_data": {
        "rule": {"id": 100001},
        "stats": {"event_count_5m": 150, "unique_ips": 50},
        "context": {"asset_criticality": "low"}
      },
      "enrichment_data": {
        "vt_score": 5
      }
    },
    {
      "event_id": "important-alert-001",
      "source": "wazuh",
      "timestamp": "2024-01-15T10:35:00Z",
      "description": "Multiple SSH authentication failures",
      "severity": 8,
      "host_context": {
        "hostname": "prod-app-01",
        "ip_address": "192.168.1.100"
      },
      "raw_data": {
        "rule": {"id": 5710},
        "stats": {"event_count_5m": 5, "unique_ips": 1},
        "context": {"asset_criticality": "high"}
      },
      "enrichment_data": {
        "vt_score": 75
      }
    }
  ]
}'
```

EXAMPLE RESPONSE:

```
[
  {
    "alert_id": "noise-alert-001",
    "classification": "noise",
    "confidence": 0.88,
    "summary": "Rule 100001 marked as noise"
  },
  {
    "alert_id": "important-alert-001",
    "classification": "important",
    "confidence": 0.95,
```

```
    "summary": "Rule 5710 marked as important"
  }
]
```

FEATURES USED FOR CLASSIFICATION:

- 1. rule_id: Wazuh rule identifier (known noisy vs. important rules)
- 2. severity: Alert severity level (1-15)
- 3. description: Alert description text
- 4. event_count_5m: Number of events in last 5 minutes (high = likely noise)
- 5. unique_ips: Number of unique source IPs (high = likely noise/spam)
- 6. ip_reputation: VirusTotal score (high = suspicious)
- 7. asset_criticality: Criticality of the affected asset (high/medium/low)

CLASSIFICATION LOGIC:

- High event_count_5m (>50): Likely noise from repeated scanning
- High unique_ips (>10): Likely mass scanning/spamming
- High IP reputation (>50): Indicates malicious activity
- High asset_criticality + low noise indicators: Important
- Low severity + high noise indicators: Noise

3.5 PLAYBOOKS - TEAM 4: PLAYBOOKS

ENDPOINT: POST /api/v1/playbooks/decision

DESCRIPTION:

Unified decision engine combining tag-based logic with risk-score thresholds to recommend automated or manual actions.

REQUEST BODY:

```
{
  "alert": UnifiedAlert
}
```

EXAMPLE REQUESTS:

```
# Example 1: High-risk malware alert
curl -X POST "http://localhost:8000/api/v1/playbooks/decision" \
-H "Content-Type: application/json" \
-d '{
  "alert": {
    "event_id": "malware-001",
    "source": "wazuh",
    "timestamp": "2024-01-15T10:30:00Z",
    "description": "Trojan detected",
    "severity": 15,
    "host_context": {
      "hostname": "workstation-john",
      "ip_address": "192.168.5.100"
    },
    "raw_data": {},
    "enrichment_data": {
      "vt_score": 100,
      "risk_score": 98
    }
  }
}'

# Response
{
  "action": "QUARANTINE_HOST",
  "confidence": 0.95,
  "automation_level": "AUTO",
  "reason": "Critical severity with very high risk score (98) - immediate quarantine recommended"
}
```

```
# Example 2: Low-risk informational alert
curl -X POST "http://localhost:8000/api/v1/playbooks/decision" \
```



```
-H "Content-Type: application/json" \
-d '{
  "alert": {
    "event_id": "info-001",
    "source": "wazuh",
    "timestamp": "2024-01-15T10:30:00Z",
    "description": "User logged in",
    "severity": 3,
    "host_context": {
      "hostname": "dev-server",
      "ip_address": "192.168.10.20"
    },
    "raw_data": {},
    "enrichment_data": {
      "vt_score": 0,
      "risk_score": 15
    }
  }
}'
```

```
# Response
{
  "action": "LOG_AND_MONITOR",
  "confidence": 0.90,
  "automation_level": "MANUAL",
  "reason": "Low severity and low risk score - informational only, no action required"
}
```

AVAILABLE ACTIONS:

Action	Description
QUARANTINE_HOST	Isolate affected host from network
BLOCK_IP	Block malicious IP at firewall/IDS
ISOLATE_SERVICE	Isolate compromised service/container
INVESTIGATE	Initiate deeper forensic investigation
CREATE_TICKET	Create alert ticket for analyst review
SEND_SOC_ALERT	Alert SOC team via email/Slack/PagerDuty
RUN_PLAYBOOK	Execute automated response playbook
LOG_AND_MONITOR	Log only, continue monitoring
NO_ACTION	No action required

AUTOMATION LEVELS:

- AUTO: Fully automated action
- SEMI: Automatic with human approval required
- MANUAL: Analyst review required

DECISION LOGIC:

1. If risk_score >= 80:
 - AUTOMATION_LEVEL: AUTO
 - Action: QUARANTINE_HOST (if critical severity)
2. If risk_score >= 60:
 - AUTOMATION_LEVEL: SEMI
 - Action: BLOCK_IP or INVESTIGATE
3. If tags contain "malware" or "ransomware":
 - AUTOMATION_LEVEL: AUTO
 - Action: QUARANTINE_HOST
4. If tags contain "phishing":
 - AUTOMATION_LEVEL: SEMI
 - Action: CREATE_TICKET
5. If severity <= 3 and risk_score <= 20:
 - AUTOMATION_LEVEL: MANUAL
 - Action: LOG_AND_MONITOR

3.6 THREAT INTEL - TEAM 5: THREAT INTEL

ENDPOINT A: POST /api/v1/threat-intel/lookup

DESCRIPTION:
Looks up IP addresses or other IOCs in external threat intelligence sources (VirusTotal, AbuseIPDB, MISP) and returns aggregated results.

REQUEST BODY:

```
{
  "alert": UnifiedAlert
}
```

EXAMPLE REQUEST:

```
curl -X POST "http://localhost:8000/api/v1/threat-intel/lookup" \
-H "Content-Type: application/json" \
-d '{
  "event_id": "intel-lookup-001",
  "source": "wazuh",
  "timestamp": "2024-01-15T10:30:00Z",
  "description": "Suspicious outbound connection",
  "severity": 8,
  "host_context": {
    "hostname": "prod-server",
    "ip_address": "185.220.101.1"
  },
  "raw_data": {},
  "enrichment_data": {}
}'
```

EXAMPLE RESPONSE:

```
{
  "ioc": "185.220.101.1",
  "malicious": true,
  "reputation": 12,
  "sources": ["VirusTotal", "AbuseIPDB", "MISP"]
}
```

RESPONSE DETAILS:

- ioc: The indicator of compromise that was looked up
- malicious: Boolean indicating if any source flagged as malicious
- reputation: Composite score (0-100, lower = more malicious)
- sources: List of sources that provided positive matches

VIRUSTOTAL SCORE CALCULATION:

- percentage = (malicious detections / total detections) * 100
- Example: 65 malicious, 20 suspicious, 15 harmless = 65/100 * 100 = 65%

ABUSEIPDB SCORE:

- Direct confidence score from AbuseIPDB (0-100)
- Score > 25 typically indicates malicious activity

MISP MATCHES:

- Any match in MISP automatically sets malicious=true, reputation=0

PRIVATE IP HANDLING:

- Private IP ranges (10.x.x.x, 192.168.x.x, 172.16-31.x.x) are not looked up in external sources
 - Returns: malicious=false, reputation=100, sources=[]
 - CVEs are still extracted and scored for private IPs
-

ENDPOINT B: POST /api/v1/threat-intel/misp-sync

DESCRIPTION:
Synchronizes recent OpenSearch alerts with MISP to find existing threat intelligence matches. Fetches last 50 alerts and queries MISP for each unique IP address.

REQUEST BODY:
None required (empty POST)

EXAMPLE REQUEST:

curl -X POST "http://localhost:8000/api/v1/threat-intel/misp-sync"

EXAMPLE RESPONSE:

```
{
  "status": "success",
  "synced_events": 3,
  "events": [
    {
      "event_id": "wazuh-alerts-2024.01.15-001",
      "ip": "192.168.1.50",
      "misp_matches": 5
    },
    {
      "event_id": "wazuh-alerts-2024.01.15-002",
      "ip": "10.0.0.25",
      "misp_matches": 2
    },
    {
      "event_id": "wazuh-alerts-2024.01.15-003",
      "ip": "185.220.101.45",
      "misp_matches": 12
    }
  ]
}
```

RESPONSE DETAILS:

- status: "success" or "error"
- synced_events: Number of events with at least one MISP match
- events: Array of events with MISP matches
 - event_id: OpenSearch event identifier
 - ip: Source IP that was queried
 - mips_matches: Number of MISP events matching this IP

3.7 HEALTH CHECK

ENDPOINT: GET /health

DESCRIPTION:
Checks connectivity to core services (OpenSearch and MISP). Returns 200 OK if healthy, 503 Service Unavailable if a critical service is unreachable.

EXAMPLE SUCCESS RESPONSE:

```
{
  "status": "healthy",
  "opensearch": "connected",
  "misp": "connected"
}
```

EXAMPLE FAILURE RESPONSE:

HTTP/1.1 503 Service Unavailable

```
{
  "detail": "OpenSearch (core database) is unreachable"
}
```

HEALTH CHECK LOGIC:

1. OpenSearch Check:
- GET /_cluster/health on OpenSearch

- Returns "connected" if HTTP 200

- If fails: returns 503 error
2. MISP Check:
- GET /users/view/me on MISP

- Returns "connected" if HTTP 200

- If fails: "disconnected" (non-critical, warning only)

3.8 ROOT ENDPOINT

ENDPOINT: GET /

DESCRIPTION:
Simple endpoint to verify the API is running.

EXAMPLE RESPONSE:

```
{
  "status": "SOAR API is online",
  "version": "v1"
}
```

4. DATA MODELS - COMPLETE REFERENCE

4.1 CORE SCHEMAS

HOSTCONTEXT

Field	Type	Description
hostname	string	Hostname of the alert source
ip_address	string	Primary IP address (IPv4/IPv6)
mac_address	string null	MAC address (optional)
os_name	string null	Operating system name (optional)

ENRICHMENTDATA

Field	Type	Description
vt_score	int	VirusTotal detection percentage (0-100, 0 = clean/not scored)
abuse_score	int	AbuseIPDB confidence score (0-100) (0 = clean/not scored)
epss_score	float null	EPSS probability (0-1)
cvss_score	float null	CVSS base score (0-10)
risk_score	int null	Risk score (0-100)
misp_matches	list[string]	MISP event UUIDs
tags	list[string]	Enrichment tags (e.g., "misp_hit")
debug_info	dict	Per-service error messages

UNIFIEDALERT

Field	Type	Description
event_id	string	Unique alert identifier

source	string	Alert source (e.g., "wazuh")	
timestamp	datetime	Alert timestamp (ISO 8601)	
description	string	Human-readable alert description	
severity	int	Wazuh rule level (1-15)	
host_context	HostContext	Host information	
raw_data	dict	Original raw alert data	
enrichment_data	EnrichmentData	Enriched data (defaults provided)	
+-----+			

4.2 TEAM-SPECIFIC SCHEMAS

RISKSCOREREQUEST

```
{
  "alert": UnifiedAlert
}
```

RISKSCORERESPONSE

+-----+			
Field	Type	Description	
+-----+			
event_id	string	Alert's unique identifier	
risk_score	int	Calculated risk score (0-100)	
priority	string	Priority (LOW/MEDIUM/HIGH/CRITICAL)	
confidence	float	Confidence score (0-1)	
features	dict	Score breakdown	
+-----+			

PATCHRESPONSE

+-----+			
Field	Type	Description	
+-----+			
host	string	Affected hostname	
recommendations	list	Array of Recommendation objects	
+-----+			

RECOMMENDATION

+-----+			
Field	Type	Description	
+-----+			
cve	string	CVE identifier	
cvss	float	CVSS score	
epss	float	EPSS probability	
priority	string	Priority level	
action	string	Recommended action	
+-----+			

FILTERREQUEST

```
{
  "alerts": List[UnifiedAlert]
}
```

FILTERRESULT

+-----+			
Field	Type	Description	
+-----+			
alert_id	string	Unique alert identifier	
classification	string	"noise" or "important"	
confidence	float	Classification confidence (0-1)	
summary	string	Human-readable summary	
+-----+			

PLAYBOOKDECISIONREQUEST

```
{
  "alert": UnifiedAlert
}
```

PLAYBOOKDECISIONRESPONSE

+-----+			
Field	Type	Description	
+-----+			

action	string	Recommended action	
confidence	float	Decision confidence (0-1)	
automation_level	string	AUTO/SEMI/MANUAL	
reason	string	Human-readable explanation	
+-----+-----+-----+-----+			

INTELRESPONSE

+-----+-----+-----+-----+			
Field	Type	Description	
+-----+-----+-----+-----+			
ioc	string	Indicator of Compromise	
malicious	bool	Whether flagged as malicious	
reputation	int	Reputation score (0-100, lower = worse)	
sources	list	Sources providing positive matches	
+-----+-----+-----+-----+			

MISPSYNCRESPONSE

+-----+-----+-----+-----+			
Field	Type	Description	
+-----+-----+-----+-----+			
status	string	"success" or "error"	
synced_events	int	Number of events synced	
events	list	Array of synced events	
+-----+-----+-----+-----+			

=====

5. INTEGRATION DETAILS

=====

5.1 EXTERNAL INTEGRATIONS

VIRUSTOTAL

- Endpoint: <https://www.virustotal.com/api/v3/>
- Rate Limit: 4 requests/minute (public API)
- Data: IP reputation, malware detections, related domains
- API Key: Required in VT_API_KEY environment variable

ABUSEIPDB

- Endpoint: <https://api.abuseipdb.com/api/v2/>
- Rate Limit: Varies by subscription tier
- Data: IP reputation, abuse reports, country blocking
- API Key: Required in ABUSEIPDB_KEY environment variable

MISP (Malware Information Sharing Platform)

- Endpoint: Configured MISP instance URL
- Features: IOC sharing, threat actor tracking, galaxy clusters
- Authentication: API key in header

NVD (National Vulnerability Database)

- Endpoint: <https://services.nvd.nist.gov/rest/json/cves/2.0>
- Data: CVE details, CVSS scores, affected software
- Rate Limit: 6 requests per second (6 calls in 10 sec window)

EPSS (Exploit Prediction Scoring System)

- Endpoint: <https://api.first.org/data/v1/epss>
- Data: Probability of exploitation in next 30 days
- Update: Daily data feed

OPENSEARCH

- Default Index: wazuh-alerts-*
- Authentication: Basic auth (OS_USER, OS_PASS)
- Connection: HTTP (non-SSL by default)

5.2 PARALLEL PROCESSING

The API implements parallel processing at multiple levels:

Level 1: Batch Alert Processing

- Uses `asyncio.gather()` to process multiple alerts concurrently

- return_exceptions=True prevents one failure from stopping the batch

Level 2: Per-Alert Enrichment

- VT, AbuseIPDB, and MISP lookups run in parallel
- Each service has 5-second timeout

Level 3: CVE Score Fetching

- NVD and EPSS lookups run in parallel for all CVEs found

5.3 FAIL-SOFT ARCHITECTURE

The system is designed with fail-soft principles:

1. Service Timeouts: 5 second max per external service call
2. Exception Handling: return_exceptions=True in gather calls
3. Safe Defaults:
 - Internal IPs: vt_score=0, abuse_score=100
 - Failed lookups: keep existing values or use defaults
4. Debug Info: All errors captured in enrichment_data.debug_info

=====

6. ADVANCED FEATURES

=====

6.1 ENRICHMENT PIPELINE

The enrichment pipeline executes in this order:

1. IP Enrichment (parallel)
 - VirusTotal IP lookup
 - AbuseIPDB IP lookup
 - MISP IP search
2. CVE Extraction
 - Regex scan of description and raw_data
 - Pattern: CVE-YYYY-NNNN (or CVE-YYYY-NNNNN+)
 - Tag format: vuln:CVE-YYYY-NNNN
3. CVE Scoring (if CVEs found, parallel)
 - EPSS score lookup
 - NVD CVSS score lookup
 - Store maximum values across all CVEs
4. Tag Population
 - "misp_hit" if MISP match found
 - "vuln:CVE-*" for each CVE found

6.2 CVE EXTRACTION

The CVE extraction engine:

1. Searches alert.description and alert.raw_data for CVE pattern
2. Extracts unique CVEs found
3. Adds "vuln:CVE-YYYY-NNNN" tags
4. Queries NVD and EPSS for each CVE
5. Stores maximum CVSS and EPSS across all CVEs

Example:

Alert: "Buffer overflow in Apache 2.4.49 - CVE-2021-41773 and CVE-2021-42013"

-> Tags: ["vuln:CVE-2021-41773", "vuln:CVE-2021-42013"]

-> epss_score: 0.98 (max)

-> cvss_score: 9.8 (max)

6.3 RISK SCORING ALGORITHM

Detailed algorithm breakdown:

```
BASE_SCORE = severity * 10
// severity 1-15 -> base_score 10-150
```

```
ENRICHMENT_SCORE =
+ (cvss_score or 0) * 5      // max ~50
+ (epss_score or 0) * 50    // max ~50
+ (abuse_score or 0) * 0.5  // max ~50
+ (vt_score or 0) * 0.5     // max ~50
```

```
FINAL_SCORE = min(BASE_SCORE + ENRICHMENT_SCORE, 100)
```

```
PRIORITY =
  if FINAL_SCORE > 70: "CRITICAL"
  else if FINAL_SCORE > 30: "HIGH"
  else: "MEDIUM" (actually returns LOW in code)
```

6.4 PLAYBOOK DECISION LOGIC

The playbook decision engine uses a multi-factor evaluation:

- Input Factors:
- alert.severity (1-15)
 - alert.enrichment_data.risk_score (0-100)
 - alert.enrichment_data.tags
 - alert.enrichment_data.vt_score
 - alert.enrichment_data.abuse_score
 - alert.description (keyword detection)

Decision Matrix:

+-----+-----+-----+			
Condition	Automation	Action	
+-----+-----+-----+			
Risk >= 90	AUTO	QUARANTINE_HOST	
Risk 70-89	SEMI	BLOCK_IP	
Risk 50-69	SEMI	INVESTIGATE	
Risk < 50	MANUAL	LOG_AND_MONITOR	
Tag contains	AUTO	QUARANTINE_HOST	
"malware"			
Tag contains	SEMI	CREATE_TICKET	
"phishing"			
Severity >= 12	SEMI	SEND_SOC_ALERT	
+-----+-----+-----+			

=====

7. ERROR HANDLING

=====

7.1 HTTP STATUS CODES

+-----+-----+-----+			
Code	Meaning		
+-----+-----+-----+			
200	Success - Request completed normally		
400	Bad Request - Invalid query parameters or malformed JSON		
422	Unprocessable Entity - Validation error (Pydantic)		
500	Internal Server Error - Unexpected server error		
503	Service Unavailable - External dependency unreachable		
+-----+-----+-----+			

7.2 ERROR RESPONSE FORMAT

```
{
  "detail": "Error description here"
}
```

Example - OpenSearch unreachable:
HTTP 503


```
{
  "detail": "OpenSearch (core database) is unreachable"
}
```

Example - Validation error:

```
HTTP 422
{
  "detail": [
    {
      "loc": ["body", "alert", "severity"],
      "msg": "ensure this value is less than or equal to 15",
      "type": "value_error"
    }
  ]
}
```

7.3 SERVICE AVAILABILITY

Critical Services (required):

- OpenSearch: API returns 503 if unavailable at startup or health check

Non-Critical Services (optional):

- MISP: Shown as "disconnected" in health check, but API remains functional
- VirusTotal: Fail-soft, returns vt_score=0 on failure
- AbuseIPDB: Fail-soft, returns abuse_score=0 on failure
- NVD: Fail-soft, cvss_score remains None on failure
- EPSS: Fail-soft, epss_score remains None on failure

8. TROUBLESHOOTING

8.1 COMMON ISSUES

Issue: "Missing mandatory environment variable: OS_HOST"
Solution: Ensure OS_HOST is set in environment variables

Issue: "OpenSearch (core database) is unreachable" (503)
Solution:
- Verify OpenSearch is running
- Check OS_HOST, OS_PORT, OS_USER, OS_PASS settings
- Verify network connectivity to OpenSearch

Issue: Enrichment returns vt_score=0, abuse_score=100
Cause: Likely a private/internal IP (skipped external lookups)
Solution: This is expected behavior for non-routable IPs

Issue: "VT exception: 403 Forbidden"
Solution: Verify VT_API_KEY is valid and has API access

Issue: "MISP connection refused"
Solution:
- Verify MISP_URL is correct
- Verify MISP_KEY is valid
- Check network connectivity to MISP instance

Issue: Slow response times with enrich=true
Cause: Parallel enrichment processes up to 100 alerts simultaneously
Solution:
- Reduce limit parameter
- External services may be slow (5s timeout each)

8.2 DEBUG INFORMATION

Enrichment errors are stored in alert.enrichment_data.debug_info:

```
{
```

```
"enrichment_data": {
  "debug_info": {
    "vt_error": "VT exception: ...",
    "abuse_error": "AbuseIPDB exception: ...",
    "misp_error": "MISP exception: ...",
    "epss_error_cve-2024-1234": "...",
    "nvd_parse_error_cve-2024-5678": "..."
  }
}
```

- Logs are available with prefix markers:
- [Alerts]: Alert fetching and processing
 - [Intel]: Threat intelligence enrichment
 - [CVE]: CVE extraction and scoring
 - [Monitor]: Incoming/outgoing service calls

=====

9. RATE LIMITS & BEST PRACTICES

=====

RATE LIMITS BY SERVICE:

+-----+-----+-----+			
Service	Rate Limit	Notes	
+-----+-----+-----+			
VirusTotal	4/minute	Public API, consider caching	
AbuseIPDB	Variable	Depends on subscription tier	
NVD	6/second	NIST official limits	
EPSS	None	Free API, updated daily	
MISP	Variable	Depends on instance configuration	
+-----+-----+-----+			

BEST PRACTICES:

1. Pagination: Use offset parameter for large result sets
2. Enrichment: Use sparingly - each alert makes up to 3 external calls
3. Caching: Consider implementing caching for repeat lookups
4. Timeouts: Default 5s per service call - adjust if needed
5. Monitoring: Check /health endpoint for service status

=====

END OF DOCUMENTATION

=====